SpoutPlugin

Unreal Engine 5 Spout Sender / Receiver Plugin

User Manual

Plugin version	1.0.0 (beta)
Engine target	Unreal Engine 5.3 - 5.8
Module	SpoutPlugin (Runtime)
Platform	Windows 64-bit, DirectX 12 RHI
Documentation date	2026-08-03

GPU texture sharing between Unreal Engine and external real-time applications such as TouchDesigner, via a D3D11-on-D3D12 bridge.

Table of Contents

About This Manual 3

1. What SpoutPlugin Is 3

2. Requirements 3

3. Installation and Project Setup 4

- Precompiled binary . . . 4
- Source build . . . 4
- Set DirectX 12 . . . 5
- Enable the plugin . . . 5
- Verify it loaded . . . 5

4. Quick Start: Sending the Game Viewport 6

5. Quick Start: Sending a Render Target 7

6. Quick Start: Receiving a Spout Stream 8

7. Sender Workflow 9

8. Receiver Workflow 10

9. SceneCapture2D and Render Target Notes 11

10. Materials 11

11. Blueprint API Reference 12

12. Example Blueprint Setups 14

13. Packaging and Distribution 15

14. Performance Notes 15

15. Limitations 16

16. Troubleshooting 16

17. Best Practices 17

18. Appendix 17

About This Manual

This manual explains how to use SpoutPlugin in Unreal Engine 5, mainly through Blueprints. It is written for Unreal users and technical artists. You do not need to write C++ to use the plugin. Short function signatures appear only in the Blueprint API reference, as the source of truth for each node's inputs and outputs.

About the screenshots

Because the plugin is driven almost entirely from Blueprint, this manual relies on screenshots of the node graphs and editor settings. Where a screenshot has not been added yet, you will see a labelled placeholder box that names the exact file to provide and describes what the image must show.

To add the images: capture each screenshot, name it exactly as the placeholder states, and place it in the folder below. Then rebuild the PDF and each placeholder is replaced automatically by the embedded image with a numbered caption.

```
Docs/manual_screenshots/ (PNG or JPG, named per the placeholder, for example
SP-005_sender-viewport-graph.png)
```

Figure IDs (SP-001, SP-002, ...) are stable. A full list with capture notes is in Docs/SCREENSHOT_CHECKLIST.md. Optional figures are marked in their capture notes.

1. What SpoutPlugin Is

Spout is a Windows technology for sharing GPU textures between real-time applications with no disk or network transfer. One application publishes a texture (a **sender**) and another reads it live (a **receiver**). It is widely used to move video between tools such as TouchDesigner, Resolume, Notch, and OBS.

SpoutPlugin brings Spout into Unreal Engine 5. It sends Unreal's rendered output, or any render target, to other Spout applications, and receives Spout streams back into Unreal as textures and materials. Everything stays on the GPU.

The technical problem it solves: Spout shares DirectX 11 textures, but modern UE5 runs on the DirectX 12 RHI. The plugin stands up a D3D11-on-D3D12 interop device and bridges UE's D3D12 textures to Spout's D3D11 shared textures. You do not interact with any of this directly - it is why DirectX 12 is required and why the plugin is Windows-only.

Typical use cases

- Send an Unreal scene or camera into TouchDesigner for compositing, projection mapping, or LED output.
- Receive a live feed from another tool into Unreal and map it onto a mesh or use it in a material.
- Build a real-time pipeline between Unreal and a media-server or VJ setup.

What it does and does not do

Does: GPU texture send and receive over Spout, from the game viewport or a render target, into a texture, a dynamic material instance, or an optional render target.

Does not: it is not a video codec, not a network streaming solution, and not a color-management system. It does not apply gamma or color correction on send. It is Windows-only and requires DirectX 12.

2. Requirements

- Windows 64-bit.

- Unreal Engine 5 (5.3 through 5.8).
- DirectX 12 RHI enabled. The plugin creates a D3D11-on-D3D12 device and will not initialize on any other RHI; it logs `SpoutPlugin requires DX12 RHI.` and stops.
- `Spout.dll`, bundled under `ThirdParty/Spout/lib/amd64/` and staged to the plugin's `Binaries/Win64/` at build time.
- For source builds: Visual Studio 2022 with the C++ toolchain and a matching Windows SDK.
- The plugin is project-local. Install it into your project's `Plugins/` folder, not the engine plugins folder.

Both applications sharing a stream must run on the same GPU. Spout's shared-texture model does not span GPUs.

3. Installation and Project Setup

Precompiled binary

1. Download the ZIP matching your exact engine version (prebuilt binaries are on Patreon).
2. Extract the `SpoutPlugin` folder into `MyProject/Plugins/`.
3. Confirm `Spout.dll` is at `Plugins/SpoutPlugin/Binaries/Win64/Spout.dll`.
4. Set the project RHI to DirectX 12 (below).
5. Open the project. If Unreal offers to rebuild because of an engine-version mismatch, use the correct ZIP or build from source.

SCREENSHOT PLACEHOLDER • FIGURE SP-003

provide file: `Docs/manual_screenshots/SP-003_plugin-folder.png`

image not added yet - drop this file in and rebuild

What this figure must show: The SpoutPlugin folder inside the project Plugins directory.
- Open Windows File Explorer at MyProject/Plugins/SpoutPlugin/.
- Show the top-level folders (Binaries, Content, Resources, Source, ThirdParty) and SpoutPlugin.uplugin in the same view.

Source build

1. Copy `SpoutPlugin` into `MyProject/Plugins/`.
2. Set the project RHI to DirectX 12.
3. Generate Visual Studio project files from the `.uproject`.
4. Build from Visual Studio, or let the editor compile on open.

`Spout.dll` is staged automatically to the plugin's `Binaries/Win64` via Unreal's runtime dependency system. Do not copy it by hand.

SCREENSHOT PLACEHOLDER • FIGURE SP-004

provide file: `Docs/manual_screenshots/SP-004_spout-dll-binaries.png`

image not added yet - drop this file in and rebuild

What this figure must show: Spout.dll staged next to the module binary in Binaries/Win64.
- Open File Explorer at MyProject/Plugins/SpoutPlugin/Binaries/Win64/.
- Make sure both Spout.dll and UnrealEditor-SpoutPlugin.dll are visible in the listing.

Set DirectX 12

Edit > Project Settings > Platforms > Windows > Default RHI > DirectX 12, then restart the editor.

SCREENSHOT PLACEHOLDER • FIGURE SP-002

provide file: `Docs/manual_screenshots/SP-002_rhi-directx12.png`

image not added yet - drop this file in and rebuild

What this figure must show: Default RHI set to DirectX 12 in Project Settings.
- Open Edit > Project Settings > Platforms > Windows.
- Scroll to the Targeted RHIs / Default RHI section.
- Capture the Default RHI dropdown showing DirectX 12 selected.

Enable the plugin

The descriptor enables the plugin by default. Confirm in Edit > Plugins by searching "Spout Plugin". Restart the editor if you change the enabled state.

SCREENSHOT PLACEHOLDER • FIGURE SP-001

provide file: `Docs/manual_screenshots/SP-001_plugin-enabled.png`

image not added yet - drop this file in and rebuild

What this figure must show: The Spout Plugin enabled in the Plugins browser.
- Open Edit > Plugins and type "Spout" in the search box.
- Frame the Spout Plugin tile so its name, the enabled checkbox (ticked), and the Rendering category are visible.
- Include the 'Beta' label if shown.

Verify it loaded

Open the Output Log (Window > Output Log) and look for `Spout Plugin Loaded` and, after first use of any node, `SpoutDX11on12 Initialized.`. If `Spout.dll` is missing, the plugin logs an explicit startup error naming the paths it checked.

SCREENSHOT PLACEHOLDER • FIGURE SP-013

provide file: `Docs/manual_screenshots/SP-013_log-loaded.png`

image not added yet - drop this file in and rebuild

What this figure must show: Output Log confirming the plugin loaded and initialized.
- Open Window > Output Log and filter for LogSpoutPlugin.
- Capture lines 'Spout Plugin Loaded' and 'SpoutDX11on12 Initialized.'

4. Quick Start: Sending the Game Viewport

This is the fastest way to get Unreal's rendered image into another application. It sends the game window backbuffer - exactly what you see in Play mode.

Build the Blueprint

1. Create a Blueprint Actor (or open the Level Blueprint) and open its Event Graph.
2. Right-click the canvas, search "Spout Sender", and place the node.

SCREENSHOT PLACEHOLDER • FIGURE SP-017

provide file: `Docs/manual_screenshots/SP-017_add-sender-node.png`

image not added yet - drop this file in and rebuild

What this figure must show: Adding the Spout Sender node from the Blueprint context menu.
- In a Blueprint Event Graph, right-click on empty canvas.
- Type "Spout Sender" in the action search box.
- Capture the search results showing the Spout Sender node under the Spout category before placing it.

3. Connect Event Tick to the Spout Sender execution input. Tick calls it every frame, which keeps the stream alive.
4. Set **Spout Name** to a unique, stable name, for example `UnrealViewport`. The receiver must use this exact name.
5. Set **Send Texture From** to `GameViewport`.
6. Leave **Texture Render Target 2D** empty - it is ignored in this mode.
7. If the actor is a Blueprint Actor, place one instance in the level so its graph runs.

SCREENSHOT PLACEHOLDER • FIGURE SP-005

provide file: `Docs/manual_screenshots/SP-005_sender-viewport-graph.png`

image not added yet - drop this file in and rebuild

What this figure must show: Blueprint: sending the game viewport every frame.
- Build the graph: Event Tick (or a timer) -> Spout Sender.
- Set Spout Name to a readable value such as UnrealViewport.
- Set Send Texture From to GameViewport. Leave Texture Render Target 2D empty.
- Capture the full node with all pins and values readable. Zoom so pin text is legible.

Receive it in another application

Start your external Spout receiver (for example a Spout In TOP in TouchDesigner), then choose `UnrealViewport` from its sender list. You should see the live Unreal image.

SCREENSHOT PLACEHOLDER • FIGURE SP-011

provide file: `Docs/manual_screenshots/SP-011_external-app-receiving.png`

image not added yet - drop this file in and rebuild

What this figure must show: An external application receiving the Unreal stream.
- In TouchDesigner (or another Spout receiver), add a Spout In TOP.
- Select the Unreal sender by name from its sender list.
- Capture the live Unreal image in the external app with the sender name visible in the dropdown.

The viewport is captured when the window is ready to present, so the game window must be actively rendering and not minimized. If nothing appears, confirm Play is running and the name matches exactly.

Optional: send on a timer instead of every frame

If you do not need full frame-rate output, drive the sender from a looping timer instead of Tick. This reduces GPU copy cost. Put the Spout Sender call in a function or custom event and trigger it with Set Timer by Event.

SCREENSHOT PLACEHOLDER • FIGURE SP-018

provide file: `Docs/manual_screenshots/SP-018_timer-setup.png`

image not added yet - drop this file in and rebuild

What this figure must show: Driving the sender on a timer instead of Event Tick.
- Show a Set Timer by Event (or Set Timer by Function Name) node configured to call the function that runs Spout Sender.
- Set Time to your interval (for example 0.033 for ~30 fps) and Looping = true.
- Capture the timer node and the bound event/function name.

5. Quick Start: Sending a Render Target

Sending a render target gives you control over exactly what is shared - a specific camera, a clean feed without UI, or a custom resolution. The usual way to fill a render target is with a SceneCapture2D actor.

Set up the capture

1. Create a `TextureRenderTarget2D` asset. RGBA8 sRGB is the recommended format. You can also create one at runtime with the Create Texture Render Target 2D node.

SCREENSHOT PLACEHOLDER • FIGURE SP-008

provide file: `Docs/manual_screenshots/SP-008_rendertarget-settings.png`

image not added yet - drop this file in and rebuild

What this figure must show: TextureRenderTarget2D asset settings.
- Double-click the render target asset to open it, or select it and view Details.
- Capture the Render Target Format (RGBA8 sRGB recommended) and Size X / Size Y fields.

2. Add a `SceneCapture2D` actor to the level and aim it at what you want to send.

3. In the SceneCapture2D Details, set **Texture Target** to the render target from step 1.
4. Set **Capture Source** to **Final Color (HDR) in Linear Working Color Space**. Do not use SceneColor (HDR) - it sends black.

SCREENSHOT PLACEHOLDER • FIGURE SP-007

provide file: `Docs/manual_screenshots/SP-007_scenecapture-source.png`

image not added yet - drop this file in and rebuild

What this figure must show: SceneCapture2D Capture Source set correctly.
- Select the SceneCapture2D actor and open its Details panel.
- Open the Capture Source dropdown.
- Capture it showing 'Final Color (HDR) in Linear Working Color Space' selected.
- This is the most important screenshot for avoiding black output - make the value unambiguous.

Send the render target

5. On Event Tick or a timer, call **Spout Sender**.
6. Set **Send Texture From** to `TextureRenderTarget2D`.
7. Set **Texture Render Target 2D** to the same render target asset.
8. Set a **Spout Name** and view the stream in your external application.

SCREENSHOT PLACEHOLDER • FIGURE SP-006

provide file: `Docs/manual_screenshots/SP-006_sender-rendertarget-graph.png`

image not added yet - drop this file in and rebuild

What this figure must show: Blueprint: sending a TextureRenderTarget2D.
- Build the graph: Event Tick (or timer) -> Spout Sender.
- Set Send Texture From to TextureRenderTarget2D.
- Connect or select your render target asset on the Texture Render Target 2D pin.
- Capture the node with the render target reference and Spout Name visible.

If the output is black, the cause is almost always the Capture Source in step 4. See section 9 for the full black-output checklist.

6. Quick Start: Receiving a Spout Stream

Receiving pulls a stream from any Spout application into Unreal. You get a live texture, and optionally a ready-to-use material instance and a render target copy.

Build the Blueprint

1. On Event Tick or a timer, place a **Spout Receiver** node.
2. Set **Spout Name** to exactly match the sender, including case.
3. Optionally set **Input Material** to a base material and **Texture Parameter Name** to a texture parameter in it (see section 10). Leave both empty if you only need the raw texture.
4. Optionally expand the advanced pins and set **Optional Output Render Target** to copy the frame into a render target as well.

5. Read the outputs: **Out Texture** (a live UTexture2D), **Out Mat** (a dynamic material instance), and the return **bool** (true when the sender was found).

6. Branch on the return value, and validate Out Mat / Out Texture before using them - the sender may not be running yet.

7. Apply Out Mat to a mesh (Set Material), or sample Out Texture directly.

SCREENSHOT PLACEHOLDER • FIGURE SP-009

provide file: `Docs/manual_screenshots/SP-009_receiver-graph.png`

image not added yet - drop this file in and rebuild

What this figure must show: Blueprint: receiving a stream and using the outputs.
- Build the graph: Event Tick (or timer) -> Spout Receiver.
- Set Spout Name to match the sender exactly.
- Expand the advanced pins so Optional Output Render Target is visible.
- Show Out Mat, Out Texture, and the return-value pins wired to something (for example a Branch on the return bool, and Out Mat into a Set Material node).
- Capture the whole node with pin names legible.

If the receiver returns false or the image is blank, check the name match first, then confirm the sender is actually running and being called continuously.

7. Sender Workflow

The sender publishes a UE texture to other Spout applications as a shared GPU texture. Two sources are available, chosen by the Send Texture From pin.

GameViewport

Sends the active window backbuffer - what you see in Play mode. The plugin hooks the point in the frame where the window is ready to present, so the capture is correctly timed. The window must be rendering and not minimized. Leave the Texture Render Target 2D pin empty.

TextureRenderTarget2D

Sends a render target you provide. This is the flexible option: drive the render target with a SceneCapture2D, a Draw Material to Render Target call, or any other render-to-texture method. The render target is resolved on the game thread and copied on the render thread.

Stream name rules

- The name must match the receiver exactly, including case.
- Only one sender can own a name at a time. Two senders sharing a name conflict.
- Sender and receiver entries are tracked separately, so a sender and a receiver in the same project may use the same name without interfering.

Lifetime

Call the sender continuously (Tick or timer). Nothing keeps the stream alive between calls. The shared texture is created and resized automatically when the source size or format changes. Call **Close Sender** to stop a stream; it affects only the sender entry, leaving any receiver with the same name running.

Performance

- Cost scales with resolution. Use a timer when full frame rate is not needed.

- The shared texture and the interop wrap are cached and reused while size and format are stable.
- No CPU readback happens on the send path - it is a GPU-to-GPU copy.
- No sender-side colour correction is applied. What the sender advertises is decided by the render target's own gamma setting: a Force Linear Gamma target publishes a plain _UNORM format, a non-linear one publishes _UNORM_SRGB. Pair a linear sender with Color Space = Linear on the receiver.

8. Receiver Workflow

The receiver consumes a stream and exposes it to Unreal. Call Spout Receiver continuously with a matching name.

Output	Type	Notes
Out Texture	UTexture2D (by ref)	Transient texture, resized to the stream, updated each call.
Out Mat	UMaterialInstanceDynamic (by ref)	Created from Input Material on the first call, with Out Texture bound to the para
Optional Output Render Target	UTextureRenderTarget2D (input)	If provided, the frame is also copied into it; resized automatically.

Finding senders: use the **Spout Info** node to list active sender names, widths, and heights - useful for a debug overlay or a dropdown. Names must match exactly. The receiver reconnects automatically when the sender appears or changes size.

SCREENSHOT PLACEHOLDER • FIGURE SP-012

provide file: `Docs/manual_screenshots/SP-012_spout-info-debug.png`

image not added yet - drop this file in and rebuild

What this figure must show: Spout Info output listing active senders.
- Build a small debug graph that calls Spout Info and prints the resulting array (name, width, height).
- Capture either the Blueprint graph or the on-screen / log output showing one or more sender entries.

Receive paths (CVar r.Spout.GPUReceiver)

The receiver has two implementations, selected at runtime by a console variable. Both produce identical images; the difference is performance only.

Value	Path	Behavior
1 (default)	GPU-direct	Shared texture copied straight into the UE destination via D3D11-on-12. No staging texture, no
0	Legacy CPU readback	Copies to a staging texture, maps it, uploads via RHIUpdateTexture2D. For debugging; more ex

Set it in the console, in `DefaultEngine.ini` under `[SystemSettings]`, with a `-ExecCmds` launch argument, or from a Blueprint Execute Console Command. It is read once per receive call, so changes apply on the next call with no restart.

SCREENSHOT PLACEHOLDER • FIGURE SP-015

provide file: `Docs/manual_screenshots/SP-015_cvar-console.png`

image not added yet - drop this file in and rebuild

What this figure must show: Setting r.Spout.GPUReceiver from the console.
- Open the editor console (tilde key) in PIE.
- Type r.Spout.GPUReceiver and capture the printed current value, plus an example of setting it to 0 or 1.

The `OutputRenderTarget` field on FSenderStruct is unused. The functional render-target output is the Optional Output Render Target pin on the Spout Receiver node.

9. SceneCapture2D and Render Target Notes

Recommended Capture Source: Final Color (HDR) in Linear Working Color Space.

Do not use: SceneColor (HDR) in RGB - it produces a black image on the receiver. This is a known limitation, not something tunable through other Capture Source values.

Recommended render target format: RGBA8 sRGB (start here). RGB10A2 is also tested and useful to reduce banding.

The plugin shares the texture as-is and does not color-correct on send, so color correctness depends on the Capture Source and render target format. The tested combination is Final Color (HDR) in Linear Working Color Space with an RGBA8 sRGB render target.

Black-output checklist

- Capture Source is Final Color (HDR) in Linear Working Color Space.
- The render target is assigned to the SceneCapture2D Texture Target.
- The capture is active and pointed at visible geometry.
- You are sending the same render target asset you assigned to the capture.
- The render target uses a tested format.

10. Materials

The receiver can hand you a ready-to-use material instance with the live texture already bound. To use it, your material needs a Texture Parameter whose name matches what you pass to the node.

1. Create a material with a **Texture Parameter** (the example content is M_SpoutMaterial). Connect it to Base Color or Emissive.
2. Name the parameter. If you leave Texture Parameter Name empty on the node, the plugin uses `SpoutTexture`, so naming the parameter `SpoutTexture` matches the default.

SCREENSHOT PLACEHOLDER • FIGURE SP-010

provide file: `Docs/manual_screenshots/SP-010_material-texture-parameter.png`

image not added yet - drop this file in and rebuild

What this figure must show: Material with a Texture Parameter for receiver output.
- Open M_SpoutMaterial (or your own material).
- Select the Texture Parameter node and show its Parameter Name in the Details panel.
- Capture so the parameter name (default SpoutTexture) is readable and is connected to Base Color / Emissive.

3. Pass the material as **Input Material** to the Spout Receiver node, and set **Texture Parameter Name** to the parameter name.
4. On the first call with a valid Input Material, the plugin creates a dynamic material instance, binds Out Texture to the parameter, and returns it as **Out Mat**.
5. Apply Out Mat to a mesh with a Set Material node.

If you do not need a material, leave Input Material empty and use Out Texture directly, or use the Optional Output Render Target.

11. Blueprint API Reference

All nodes are static functions on `USpoutBPFunctionLibrary`, category `Spout`. The function signatures below are the source of truth; Blueprint pin names match the parameter names.

Node	Returns
Spout Sender	bool
Close Sender	void
Spout Receiver	bool
Close Receiver	void
Spout Info	bool
Get Sender Info	bool
Set Max Senders	int32
Get Max Senders	void
Create Texture Render Target 2D	UTextureRenderTarget2D*

Spout Sender

```
bool SpoutSender(FName SpoutName, ESpoutSendTextureFrom SendTextureFrom,
UTextureRenderTarget2D* TextureRenderTarget2D)
```

Publishes a texture to Spout; the GPU copy runs on the render thread. Returns true when accepted or queued. Provide the render target when using the TextureRenderTarget2D source. Call continuously. Common mistakes: empty render target with the render-target source; calling once; name mismatch.

Close Sender

```
void CloseSender(FName SpoutName)
```

Removes the named sender and releases its handle. Scoped to senders only, so a receiver using the same name is unaffected.

Spout Receiver

```
bool SpoutReceiver(const FName SpoutName, UMaterialInterface* InputMaterial,
FName TextureParameterName, UMaterialInstanceDynamic*& OutMat,
UTexture2D*& OutTexture,
UTextureRenderTarget2D* OptionalOutputRenderTarget = nullptr,
ESpoutColorSpace ColorSpace = ESpoutColorSpace::Auto)
```

Receives a stream. OptionalOutputRenderTarget and Color Space are advanced-display pins. Returns true when the sender exists and a receive was enqueued. Outputs Out Mat and Out Texture, which are OUTPUTS and not ownership: the received texture belongs to the plugin, which keeps it alive while the receiver is open. Keep the values this node returns and pass them back on later calls and to Close Receiver. Texture Parameter Name defaults to SpoutTexture when empty. The frame is only copied when the sender published a new one.

Optional Output Render Target. When provided, the received frame is also copied into it. Its pixel format must match the incoming frame's format exactly and it must be single-sampled (no MSAA); no implicit format conversion is performed. A mismatched target is left completely untouched and the mismatch is logged - the call still succeeds and Out Texture still updates - which is what a target that stays black is telling you. Size differences are not a mismatch: the target is resized to match the stream.

Color Space. Decides how the received bytes are interpreted. **Auto** (the default) follows the format convention the Spout ecosystem publishes: 8-bit frames decode as sRGB, while 10-bit and 16-bit float frames stay linear. **Linear** applies no decoding; use it when the sender publishes linear or data values in an 8-bit format, such as a render target created with Force Linear Gamma. **sRGB** forces decoding, and is only representable for 8-bit frames: requesting it for a 10-bit or float frame fails the call and logs, rather than being silently ignored.

Close Receiver

```
void CloseReceiver(FName SpoutName, UTexture2D*& ReceivedTexture,
UMaterialInstanceDynamic*& ReceivedMaterial)
```

Closes the receiver session for that name. There is one session per sender name, so this closes it for every graph receiving it, and it detaches the Spout texture parameter from every material the plugin bound it into. The NAME decides what is released, not the pointers: pass back the Out Texture and Out Mat this plugin gave you and both are cleared; pass anything else and it is left untouched with a warning. This node will not destroy a texture it did not create.

Spout Info

```
bool SpoutInfo(TArray<FSenderStruct>& Senders)
```

Fills an array with one entry per active sender. Only Name, Width, Height, and bIsAlive are populated. Returns true if the registry was queried.

Get Sender Info

```
bool GetSenderInfo(FName SpoutName, FSenderStruct& OutSenderStruct)
```

Looks up one sender by name; fills Name, Width, Height, bIsAlive. Returns true if it exists.

Set Max Senders

```
int32 SetMaxSenders(int32 Max)
```

Sets the maximum sender count in Spout's registry and returns the limit actually in effect. Returns 0 when nothing changed, either because the requested value was outside the accepted range (1-1024) or because Spout is unavailable. It does not echo the requested value back.

Get Max Senders

```
void GetMaxSenders(int32& Max)
```

Reads the current maximum sender count into Max.

Create Texture Render Target 2D

```
UTextureRenderTarget2D* CreateTextureRenderTarget2D(int32 Width = 1024, int32 Height = 768,
EPixelFormat Format = PF_B8G8R8A8, bool bForceLinearGamma = true)
```

Creates a render target at runtime, sets wrap addressing on both axes, and initializes its resource. The result is NOT rooted: keep a reference to it exactly as you would with Create Render Target 2D, or it will be collected. Returns null for invalid dimensions or for a pixel format the plugin cannot share (supported: B8G8R8A8, R8G8B8A8, A2B10G10R10, FloatRGBA). Force Linear Gamma decides what colour space the sender advertises for 8-bit formats: true publishes a plain _UNORM (pair it with Color Space = Linear on the receiver), false publishes _UNORM_SRGB.

Enums

ESpoutType: Sender, Receiver. **ESpoutSendTextureFrom:** GameViewport, TextureRenderTarget2D.

ESpoutColorSpace: Auto (follow the format convention - 8-bit decodes as sRGB, 10-bit and float stay linear), Linear (no decoding), SRGB (force sRGB decoding; representable for 8-bit formats only, and refused otherwise).

Supported formats. The plugin shares B8G8R8A8, R8G8B8A8, A2B10G10R10 and FloatRGBA, including their typeless variants. Anything else is refused: a warning naming the advertised format is written to the log and nothing is published or received for that frame. Unsupported formats are never reinterpreted as B8G8R8A8 and no fallback conversion is fabricated, because advertising a layout the source does not have produces silently garbled frames rather than an honest failure.

Struct: FSenderStruct

Field	Type	Filled by plugin?
Name	FName	Yes (Info nodes)
bIsAlive	bool	Yes (Info nodes)
SpoutType	ESpoutType	No
Width	int32	Yes (Info nodes)
Height	int32	Yes (Info nodes)
TextureColor	UTexture2D*	No
MaterialInstanceColor	UMaterialInstanceDynamic*	No
UserBaseMaterial	UMaterialInterface*	No
UserTextureParameter	FName (default SpoutTexture)	No
OutputRenderTarget	UTextureRenderTarget2D* (unused)	No

Rely only on Name, Width, Height, and bIsAlive from the Info nodes. The remaining fields are available for your own Blueprint use but are not driven by the plugin.

12. Example Blueprint Setups

The plugin ships example content under its `Content/` folder. These are the actual asset names; open them to see working graphs rather than building from scratch.

Asset	Purpose
BP_SpoutSenderExample_	Example sender setup.
BP_SpoutReceiverExample	Example receiver setup.
M_SpoutMaterial	Material with a texture parameter for receiver output.
RT_SpoutSenderTest_	Example render target for sending.
RT_SpoutReceiverTest	Example render target for receiving.

Enable 'Show Plugin Content' in the Content Browser settings to see them.

SCREENSHOT PLACEHOLDER • FIGURE SP-016

provide file: `Docs/manual_screenshots/SP-016_example-content.png`

image not added yet - drop this file in and rebuild

What this figure must show: The plugin example content folder.
- In the Content Browser, enable 'Show Plugin Content' and open the SpoutPlugin content folder.
- Capture the assets: BP_SpoutSenderExample_, BP_SpoutReceiverExample, M_SpoutMaterial, RT_SpoutSenderTest_, RT_SpoutReceiverTest.

13. Packaging and Distribution

`Spout.dll` ends up in a different place depending on how the plugin is being used, and conflating them is a common source of "module could not be loaded" reports. In an editor or other modular build, RuntimeDependencies stages it to the plugin's own `Binaries/Win64`. In a packaged game the plugin module is linked into the game executable, so the same `$(BinaryOutputDir)` target resolves to the PROJECT's `Binaries/Win64` beside the .exe, and the plugin's own Binaries folder is not staged at all. Raw `RunUAT BuildPlugin -Package` output does NOT receive the DLL from RuntimeDependencies; it stays under `ThirdParty/Spout/lib/amd64`, which is why `build.ps1` copies it into the release archive explicitly. The DLL is delay-loaded, and the module loads it by full path at startup so the delay-load thunks bind by name.

A release folder contains Binaries (with `Spout.dll`), Content, Resources, Source, and `SpoutPlugin.uplugin`. Precompiled releases must include `Spout.dll` in `Binaries/Win64`.

Releases are built per engine version with `RunUAT BuildPlugin ... -TargetPlatforms=Win64 -Rocket`. Debug symbols are removed, but `Intermediate/Build` is deliberately KEPT: it carries the precompiled UnrealGame Development and Shipping objects and the `.precompiled` manifests that let Blueprint-only users package a project without Visual Studio. Removing it makes such packaging impossible. Each archive is then reopened and validated before it is published. The repository `build.ps1` does this for UE 5.3 through UE 5.8. A binary built for one engine version will not load in another.

Clean rebuild after an engine upgrade

```
Project/Binaries
Project/Intermediate
Plugins/SpoutPlugin/Binaries
Plugins/SpoutPlugin/Intermediate
```

Delete those, regenerate project files, and rebuild from scratch.

14. Performance Notes

- Cost scales with texture resolution on both send and receive. Start lower and raise as needed.
- Use a timer instead of Tick when you do not need full frame-rate output.
- Leave `r.Spout.GPUReceiver` at its default of 1. The legacy path (0) adds a full-frame copy, map, and upload each frame.
- The send path is a GPU copy through the interop device; there is no CPU readback on send.
- Keep per-frame Blueprint logic around the send and receive calls minimal.
- Watch large textures: a 4K stream at full frame rate is far more expensive than 1080p.

15. Limitations

- Windows 64-bit only.
- DirectX 12 RHI required; the plugin refuses to initialize otherwise.
- Both ends must be on the same GPU.
- Tested render target formats are limited to RGBA8 sRGB and RGB10A2.
- No colour conversion on send. The advertised colour space follows the render target's gamma setting; see section 6.
- SceneCapture2D must use Final Color (HDR) in Linear Working Color Space; SceneColor (HDR) is black.
- One sender per name.
- Senders and receivers must be called continuously.
- Spout Info and Get Sender Info populate only Name, Width, Height, bIsAlive.
- Marked beta; not guaranteed production-ready without testing. Error handling favors visible, logged failures over silent recovery.

16. Troubleshooting

Symptom	What to do
Plugin does not compile	Confirm it is in the project Plugins folder, you are on UE 5.3-5.8 with VS 2022; regenerate project files
Module could not be loaded	Binary must match your exact engine; Spout.dll must be in Binaries/Win64. Read the log for the startu
Spout.dll missing	Copy it from ThirdParty/Spout/lib/amd64/ to Binaries/Win64/, or rebuild from source.
Sender not visible externally	Confirm DirectX 12; confirm continuous calls; one sender per name; for GameViewport the window mu
Receiver returns invalid output	Exact name match; check the return value; validate outputs; texture parameter name must match the
Black texture from SceneCapture2D	Set Capture Source to Final Color (HDR) in Linear Working Color Space.
Wrong colors or gamma	Adjust render target format and Capture Source; try RGBA8 sRGB first, then RGB10A2.
Stream name mismatch	Names are case-sensitive; use Spout Info to read exact names.
Multiple senders conflict	Unique, stable names per stream.
Performance drops	Keep the GPU receiver default; reduce resolution or update rate; prefer timers.
Crashes or instability	Update GPU drivers; avoid swapping render target resources at runtime; test resolutions; reproduce in
External app cannot see stream	Confirm Spout/format support and that both apps are on the same GPU.
After an engine upgrade	Clean rebuild (delete the four Binaries/Intermediate folders), regenerate, rebuild.

If you can reproduce a missing-DLL failure, the startup log line is a useful diagnostic to keep:

SCREENSHOT PLACEHOLDER • FIGURE SP-014

provide file: `Docs/manual_screenshots/SP-014_log-dll-missing.png`

image not added yet - drop this file in and rebuild

What this figure must show: Output Log showing a missing Spout.dll startup error.
- Reproduce by temporarily removing Spout.dll, or capture from a past failure.
- Capture the LogSpoutPlugin error naming the Binaries/Win64 and ThirdParty paths it checked.
- Optional figure - include only if you can reproduce it cleanly.

17. Best Practices

- Use stable, unique Spout names and keep sender and receiver names identical.
- Start at a low resolution and raise it only when needed.
- Use a timer when full frame rate is not required.
- Always validate Out Mat and Out Texture before applying them.
- Keep the render target format simple: RGBA8 sRGB first.
- For SceneCapture2D sends, always use Final Color (HDR) in Linear Working Color Space.
- Test in a minimal project before integrating into production.
- Record the engine version and the plugin binary version together; binaries are version-specific.

18. Appendix

File structure

```
SpoutPlugin.uplugin
Source/SpoutPlugin/
SpoutPlugin.Build.cs
Public/ SpoutBPFunctionLibrary.h, SpoutModule.h
Private/ SpoutBPFunctionLibrary.cpp, SpoutModule.cpp,
SpoutSender.*, SpoutReceiver.*, SpoutD3DContext.*,
SpoutD3DUtils.*, SpoutSenderRegistry.*, SpoutTextureUtils.*
ThirdParty/Spout/ include/, lib/amd64/ (Spout.dll, Spout.lib)
Content/ example Blueprints, material, render targets
Resources/ Icon128.png
Docs/ documentation and this manual
build.ps1, build.bat
```

Module and dependency summary

Single Runtime module `SpoutPlugin`, Win64 only. Public dependencies: Core, CoreUObject, Engine, RHI, RenderCore. Private: ApplicationCore, Slate, SlateCore, Projects, D3D12RHI, D3D11RHI, plus the engine DX12 and DX11 third-party libraries. Spout.dll is delay-loaded and staged as a runtime dependency.

Third-party attribution

- Spout: http://spout.zeal.co/
- Based on the original repository by zuyi53: https://github.com/zuyi53/Spout-UE5

License

MIT License. Commercial use is allowed, but reselling the plugin as-is as a standalone product goes against the spirit of the project.

Glossary

Term	Meaning
Spout	Windows GPU texture-sharing technology between real-time applications.
Sender	An application publishing a shared texture.
Receiver	An application reading a shared texture.
Render Target	A UTextureRenderTarget2D; a texture you can render into and read back.
SceneCapture2D	An Unreal actor that renders a view into a render target.
RHI	Rendering Hardware Interface, Unreal's graphics API abstraction.
D3D12	DirectX 12, the graphics API Unreal uses; required by this plugin.
D3D11-on-D3D12	An interop layer exposing D3D12 resources to D3D11 code (here, Spout).
Shared texture	A GPU texture exposed via a shared handle so another process can read it.

Figure index

Figure	File	Shows
SP-001	SP-001_plugin-enabled.png	The Spout Plugin enabled in the Plugins browser.
SP-002	SP-002_rhi-directx12.png	Default RHI set to DirectX 12 in Project Settings.
SP-003	SP-003_plugin-folder.png	The SpoutPlugin folder inside the project Plugins directory.
SP-004	SP-004_spout-dll-binaries.png	Spout.dll staged next to the module binary in Binaries/Win64.
SP-017	SP-017_add-sender-node.png	Adding the Spout Sender node from the Blueprint context menu.
SP-005	SP-005_sender-viewport-graph.png	Blueprint: sending the game viewport every frame.
SP-018	SP-018_timer-setup.png	Driving the sender on a timer instead of Event Tick.
SP-006	SP-006_sender-rendertarget-graph.png	Blueprint: sending a TextureRenderTarget2D.
SP-007	SP-007_scenecapture-source.png	SceneCapture2D Capture Source set correctly.
SP-008	SP-008_rendertarget-settings.png	TextureRenderTarget2D asset settings.
SP-009	SP-009_receiver-graph.png	Blueprint: receiving a stream and using the outputs.
SP-010	SP-010_material-texture-parameter.png	Material with a Texture Parameter for receiver output.
SP-011	SP-011_external-app-receiving.png	An external application receiving the Unreal stream.
SP-012	SP-012_spout-info-debug.png	Spout Info output listing active senders.
SP-013	SP-013_log-loaded.png	Output Log confirming the plugin loaded and initialized.
SP-014	SP-014_log-dll-missing.png	Output Log showing a missing Spout.dll startup error.
SP-015	SP-015_cvar-console.png	Setting r.Spout.GPUReceiver from the console.
SP-016	SP-016_example-content.png	The plugin example content folder.